isec Engenharia

Departamento de Engenharia Informática e de Sistemas
Licenciatura em Engenharia Informática
Licenciatura em Engenharia Informática - PL
Ramo de Desenvolvimento de Aplicações

Programação Avançada 2025/2026

Trabalho Prático



Deep Sea Mining (Java & JavaFX)

Guilherme Eça - 2024143849
Luana Rabello - 2024129510
Miguel Zara - 2024143873

Índice

Índice 2
Introdução 3
Explicação do Jogo e Opções de Design 4
Dinâmica e Fases do Jogo 4
Opções Tomadas na Implementação dos Requisitos 4
Etapa 1 - Modelagem de Dados 4
Modelação Polimórfica das Grelhas e Elementos 4
Representação Estrutural do Puzzle 15 5
Isolamento do "Modo Defesa" 5
Etapa 2 - Gestão do Jogo e Persistência Dados 5
Implementação Polimórfica da Máquina de Estados (FSM) 5
Persistência de Dados (Gravação) 5
Definição dos Estados de Oficina e Fim de Jogo como Estados Secundários da FSM 6
Etapa 3 - Interface Gráfica JavaFX 6
Construção de UI por Código e Padrão de Componentização 6
Coordenação da Área Central via GameView 6
Abstração Polimórfica dos Painéis Contextuais 7
Opção de Distribuição Espacial de Informação 7
Interface de Seleção Visual de Drones via Cartões 7
Uso Exclusivo de Canvas 7
Gestão de Recursos de Imagem Otimizada com ImageLoader (Padrão Multiton) 8
Estilização Centralizada e Dinâmica através do CSSManager 8
Notificações Assíncronas via Property Changes 8
Etapa Final - Sons e Apresentação 9
Gestor de Áudio Central 9
Fila de Reprodução Inteligente 9
Arquitetura do Sistema e Padrões de Programação 10
O Padrão MVC@PA na Aplicação 10
Model (pt.isec.pa.deepsea.model...) 10
View-Controller (pt.isec.pa.deepsea.ui...) 10
Main e MainJFX 10
Estrutura Padrão das Classes View-Controller 11
Diagrama de Estados (FSM) 12
Diagrama de Classes e Estrutura de Pacotes 13
Estrutura de Pacotes 13
Diagramas 14
Diagrama UML do Model 14
Diagrama UML do UI 15
Tabela com Resumo das Tarefas 16
Etapa 1 16
Etapa 2 16
Etapa 3 17
Etapa 4 17
Conclusão 18

Introdução

O presente relatório documenta o processo de conceção, arquitetura e implementação do projeto "Deep Sea Mining". O software desenvolvido consiste num jogo de estratégia, exploração e gestão de recursos, programado em Java, utilizando a framework JavaFX para a Interface Gráfica de Utilizador.

O principal objetivo deste projeto foi aplicar conceitos avançados de Programação Orientada aos Objetos (POO), tais como encapsulamento, polimorfismo e herança, aliados à implementação rigorosa de Padrões de Desenho de Software (*Design Patterns*) e à separação de responsabilidades ditada pela arquitetura MVC@PA.

Explicação do Jogo e Opções de Design

O "Deep Sea Mining" coloca o jogador no comando de um navio de exploração oceanográfica. A missão principal é mergulhar drones nas profundezas do oceano para recuperar um número pré-determinado de artefactos perdidos de uma civilização antiga, gerindo combustível e danos ao casco ao longo do caminho.

Dinâmica e Fases do Jogo

O jogo está dividido em 4 camadas lógicas principais:

A Superfície: Zona segura. O navio desloca-se, gere uma frota de 3 drones, e funciona como base para recarregar baterias e comprar melhorias na Oficina.

O Fosso: Uma descida (e subida) vertical perigosa. O drone enfrenta correntes (que aumentam o consumo de energia), animais marinhos e rochas (que causam dano à integridade do casco).

O Fundo do Mar: A área principal de exploração coberta por uma névoa de guerra (*Fog of War*). Aqui, o jogador movimenta-se livremente nos 4 eixos em busca de minérios e artefactos, evitando monstros errantes.

Puzzle: Fase do jogo onde o jogador tem de resolver um puzzle para coletar o artefacto que encontrou no Fundo do Mar, optamos pelo "15 Puzzle" — um tabuleiro 4×4 com 15 peças numeradas e um espaço vazio.

Opções Tomadas na Implementação dos Requisitos

Para dar resposta rigorosa às exigências dos enunciados ao longo do semestre, o grupo tomou as seguintes decisões arquiteturais na estruturação do código de cada etapa:

Etapa 1 - Modelagem de Dados

Modelação Polimórfica das Grelhas e Elementos

O enunciado exigia a aplicação de herança e polimorfismo na base do jogo. A nossa opção foi criar uma classe supertipo **`Elemento`** (da qual herdam **`Rocha`**, **`Monstro`**, **`Minerio`**, etc.). A classe **`Grelha`** utiliza matrizes bidimensionais genéricas de **`Elemento`**, permitindo que o motor do jogo resolva colisões e interações dinamicamente sem o uso abusivo de blocos **`if/else`** ou verificações de tipo

(**`instanceof`**). Além disso, as grelhas do **`Fosso`** e **`Fundo`** foram instanciadas como componentes internos geridos pela célula da **`Superficie`**, garantindo o isolamento exigido de "um fosso e fundo por zona".

Representação Estrutural do Puzzle 15

Conforme solicitado no enunciado, a justificação para a opção tomada no minijogo é a seguinte, em vez de reaproveitar a classe complexa de matriz **`Grelha`**, optou-se por desenhar o **`Puzzle`** como uma entidade completamente independente, gerindo internamente uma matriz simples de inteiros (**`int[][]`**) fixa a nxn. Esta decisão garante leveza em memória e permite isolar as regras matemáticas estritas de permutação e verificação de solubilidade, não poluindo a lógica global de exploração de mapas do jogo.

Isolamento do "Modo Defesa"

O professor exigiu a existência de uma *flag* global que facilitasse os testes na defesa. A nossa opção arquitetural foi não "sujar" o código da lógica principal com dezenas de validações **`if(ModoDefesa)`**. Criámos uma classe estritamente dedicada a isto (**`Gerador.java`**), que age como *Factory*. Quando o Modo Defesa é lido da classe **`Constantes`**, o gerador instiga coordenadas fixas, remove o fog of war e instiga apenas 1 monstro, devolvendo à matriz central um mapa formatado sem que as entidades principais do jogo tenham de conhecer as regras da avaliação.

Etapa 2 - Gestão do Jogo e Persistência Dados

Implementação Polimórfica da Máquina de Estados (FSM)

Em estrito cumprimento do requisito de gerir a evolução do jogo através de uma FSM polimórfica, o grupo implementou o padrão State. Definimos a interface **`IDeepSeaState`** e a Enumeração **`DeepSeaState`** (atuando como *Factory* de estados). O contexto FSM reside em **`DeepSeaContext`**. Para responder a este requisito de forma limpa, criamos a classe intermédia **`DeepSeaStateAdapter`**, que implementa a interface e devolve **`false`** (erro) por padrão para todas as ações. Desta forma, as classes concretas de estado (como **`OficinaState`** ou **`PuzzleState`**) herdam do adaptador e apenas reescrevem (Override) as transições que lhes competem, eliminando a duplicação de métodos inúteis.

Persistência de Dados (Gravação)

Para assegurar as funcionalidades de gravação e recuperação solicitadas, a opção tomada foi o uso de **Serialização**. Marcamos a classe de fachada **`DeepSeaContext`** (e os seus dados) com a interface **`Serializable`**. Assim, com um único método, conseguimos descarregar o estado completo

(incluindo IDs) para um ficheiro **`.bin`**, preservando na íntegra os objetos em memória no exato ponto onde a simulação foi parada.

Definição dos Estados de Oficina e Fim de Jogo como Estados Secundários da FSM

Embora o guião base se baseie essencialmente no fluxo linear do drone pelas grelhas, o grupo tomou a opção estratégica de isolar a Oficina e o encerramento da simulação em estados autónomos da Máquina de Estados (OficinaState e FimDeJogoState).

A criação do OficinaState surgiu para garantir uma separação clara de contextos e evitar que o estado Superficie ficasse sobrecarregado com excesso de informação. Em vez de acumular os controlos de navegação marítima e os botões de melhorias de drones no mesmo ecrã, o que tornaria a interface confusa e visualmente poluída, o estado de Oficina permite limpar a área central. Assim, quando o jogador decide gerir a frota, foca-se exclusivamente nisso, oferecendo assim ao utilizador uma experiência melhor.

Por sua vez, o FimDeJogoState foi concebido como um estado terminal de segurança. Sempre que as condições de derrota (combustível do navio a zero ou destruição total dos três drones) ou as condições de vitória são validadas, a FSM transita para este estado de bloqueio absoluto. Ele revoga de forma permanente qualquer processamento de inputs ou cliques do utilizador, congelando o modelo de dados em modo de leitura estática até que um novo jogo seja explicitamente inicializado.

Etapa 3 - Interface Gráfica JavaFX

Construção de UI por Código e Padrão de Componentização

Para assegurar a fácil manutenção e evitar classes gigantes de controlo visual, o grupo adotou o padrão de componentização. Dividimos a janela em classes de *View-Controller* polimórficas (**`LeftPanelStats`**, **`RightPanel`**, **`GameView`**, **`RootPane`**) que herdam diretamente de painéis nativos do JavaFX (**`BorderPane`**, **`StackPane`**, **`VBox`**). Isto possibilitou desenhar e atualizar cada área do ecrã de forma autónoma e isolada, garantindo alta coesão estrutural.

*Coordenação da Área Central via **`GameView`***

A área central do jogo é gerida pela classe **`GameView`**, que estende um **`StackPane`**. Em vez de instanciar e destruir painéis a cada transição de estado, a nossa opção técnica foi carregar todos os ecrãs do tipo **`Canvas`** (**`SuperficieCanvas`**, **`FossoCanvas`**, **`FundoCanvas`**, **`OficinaCanvas`**, **`PuzzleCanvas`**, **`FimDeJogoCanvas`**) de forma persistente dentro deste **`StackPane`**. O método **`update()`** do **`GameView`** escuta o modelo e simplesmente comuta a visibilidade (**`setVisible(true/false)`**) de cada uma das telas sobrepostas consoante o estado ativo na FSM. Esta decisão elimina totalmente o custo de processamento associado à destruição e reconstrução de nós da árvore visual no JavaFX.

Abstração Polimórfica dos Painéis Contextuais

Para o lado direito da interface, criámos a classe abstrata **`RightPanel`** (que herda de **`VBox`**) e as suas especializações contextuais (**`RightPanelSuperficie`**, **`RightPanelFosso`**, **`RightPanelFundo`**, **`RightPanelOficina`**). O **`RootPane`** troca dinamicamente a instância ativa no painel lateral direito de acordo com o estado do jogo. Esta decisão de interfaces foca a atenção do jogador apenas nos controlos específicos à sua ação ativa, ocultando botões de ação que seriam irrelevantes para o momento.

Opção de Distribuição Espacial de Informação

Painel Esquerdo Fixo (`LeftPanelStats`): Exibe estatísticas globais e persistentes da sessão (combustível do navio-mãe, frota total de drones ativos e número total de artefactos recolhidos). Mantém-se estático e inalterado na esquerda para dar ao utilizador um ponto de referência visual constante sobre o progresso geral da missão.

Painel Direito Dinâmico (`RightPanel`): Focado no contexto operacional imediato. Apresenta dados em tempo real da unidade que está em perigo ou atividade (nível do casco do drone ativo, bateria restante e inventário temporário).

Interface de Seleção Visual de Drones via Cartões

Em vez de utilizar simples botões ou caixas de seleção de texto para selecionar qual o drone a enviar ou melhorar, o grupo desenhou a classe **`DroneCardsBox`**. Esta classe desenha cartões interativos para cada um dos 3 drones da frota. Cada cartão funciona como uma *mini-view-controller* que reage a cliques e apresenta graficamente os atributos de integridade, energia e upgrades atuais de cada drone em tempo real, fornecendo um feedback visual imediato e intuitivo no hangar da Superfície e da Oficina.

Uso Exclusivo de Canvas

A Etapa 3 exigia que a representação visual central de cada momento (superfície, fossos, fundos e minijogo) recorresse estritamente a um elemento derivado de **`Canvas`** para renderizar o mapa. Cumprindo este critério, implementámos classes como **`SuperficieCanvas`** e **`FundoCanvas`**. A renderização é feita diretamente no **`GraphicsContext`** através de laços de repetição que pintam os blocos com base nas matrizes lidas do modelo. Isto evitou a sobrecarga de memória que ocorreria se tivéssemos de instanciar dezenas de nós do tipo **`ImageView`**.

*Gestão de Recursos de Imagem Otimizada com **`ImageLoader`** (Padrão Multiton)*

Como a renderização central de todos os ecrãs assenta em **`Canvas`** e redesenha os elementos gráficos ciclicamente a cada **`update()`**, seria incomportável carregar as imagens diretamente do armazenamento em disco em cada frame. Para solucionar este gargalo de desempenho e evitar travagens na Thread gráfica do JavaFX, implementou-se o **`ImageLoader`** no pacote **`pt.isec.pa.deepsea.ui.res`**. Esta classe funciona como uma cache em memória baseada num mapa (**`HashMap<String, Image>`**). Sempre que qualquer **`Canvas`** (como **`FundoCanvas`** para desenhar monstros, minerais ou artefactos, ou **`OficinaCanvas`** para os fundo do hangar) solicita uma textura, o método **`getImage(String name)`** verifica se a imagem já reside em cache. Caso sim, ela é retornada instantaneamente da RAM, caso contrário, é carregada através de **`getResourceAsStream()`** e guardada para uso futuro, otimizando drasticamente os tempos de ciclo e o consumo de processamento.

*Estilização Centralizada e Dinâmica através do **`CSSManager`***

Em conformidade com as diretivas estéticas do guião da Etapa 3, que exigiam uma interface visualmente apelativa e profissional, o grupo optou por isolar por completo as definições cosméticas da estrutura lógica das classes. Para este efeito, implementámos o utilitário **`CSSManager`** no pacote de recursos. Através do seu método estático **`applyCSS(Parent parent, String filename)`**, o ficheiro de estilo externo **`styles.css`** presente em **`/css/`** é aplicado de forma limpa nos painéis. Esta abordagem garante que elementos visuais comuns (como estilos de botões, barras de progresso customizadas, cores de fundo escuras e tipografia moderna) sejam geridos de forma desacoplada das regras de negócio, facilitando eventuais alterações estéticas globais ao longo do desenvolvimento.

Notificações Assíncronas via Property Changes

O guião impôs o uso de Property Changes para que a UI reagisse às atualizações do modelo de forma assíncrona. No **`DeepSeaManager`**, integrámos a classe **`PropertyChangeSupport`**. Sempre que ocorrem ações de movimentação, consumo ou transições de estado, o gestor emite eventos com identificadores constantes, forçando as *View-Controllers* a redesenhar-se através do método **`update()`** encapsulado de forma segura numa chamada **`Platform.runLater()`**.

Etapa Final - Sons e Apresentação

Gestor de Áudio Central

Para cumprir a integração de som proveniente de **`resources/sounds/`** utilizando o padrão Multiton/Singleton, criámos as classes **`SoundLoader`** (com cache em **`HashMap`**) e **`SoundManager`**.

Fila de Reprodução Inteligente

O enunciado exigia que movimentos rápidos anulassem sons repetidos e que a sobreposição com animais marinhos ocorresse numa sequência lógica e limpa. A nossa opção técnica foi implementar uma **Fila de Execução (`Queue<String>`)** no **`SoundManager`**. Se uma ação de movimento for desencadeada consecutivamente, o áudio anterior do motor é cortado imediatamente. No entanto, se houver um evento de colisão ou recolha, o sistema enfileira o efeito sonoro de forma assíncrona, garantindo que o áudio de sobreposição seja tocado na íntegra logo de seguida, evitando uma cacofonia desordenada.

Arquitetura do Sistema e Padrões de Programação

O projeto foi inteiramente estruturado com base na arquitetura **MVC@PA**, o modelo específico ensinado e exigido na Unidade Curricular de Programação Avançada. Esta arquitetura reconhece que, em JavaFX, muitas vezes é difícil separar a vista pura do controlador (dado que os *handlers* de eventos são configurados nos mesmos objetos que definem a interface), unificando estes conceitos num bloco de **View-Controller**.

O Padrão MVC@PA na Aplicação

*Model (**`pt.isec.pa.deepsea.model...`**)*

Aplica-se a diretiva restrita de que o Modelo deve ser completamente isolado da interface do utilizador ("neither in nor out"). Não existe qualquer dependência de classes do JavaFX dentro do pacote do modelo. Gere toda a lógica de negócio, dados e os algoritmos (Grelhas, Navio, FSM). Foi criada a classe **`DeepSeaManager`** que atua como **Facade**, escondendo os detalhes internos do modelo da View-Controller.

*View-Controller (**`pt.isec.pa.deepsea.ui...`**)*

Classes responsáveis por implementar os ecrãs. Em conformidade com o MVC@PA, estas classes (como o **`RootPane`**, **`RightPanelSuperficie`**, etc.) estendem diretamente os painéis da API JavaFX (como **`BorderPane`**, **`StackPane`**, **`VBox`**). O controlador não está numa classe separada; a lógica de gestão de interface (Handlers) e desenho visual reside na própria classe do Pane.

Main e MainJFX

Para suportar a aplicação, definiram-se as classes de entrada:

`DeepSeaApp` (Main): Contém o método genérico **`main(String[] args)`** que simplesmente chama o **`Application.launch`**.

`MainJFX`: Estende a classe **`Application`** do JavaFX. É responsável pela configuração do *Stage* e por injetar o modelo (criado ali) na primeira *View-Controller* da árvore visual (o **`RootPane`**).

Estrutura Padrão das Classes View-Controller

Para uniformizar a aplicação e respeitar os guiões do MVC@PA, todas as classes de interface (**Pane**) foram construídas seguindo uma estrutura obrigatória no seu construtor:

1. Receção da referência para o modelo (o **`DeepSeaManager`**) como parâmetro.
2. Invocação do método **`createViews()`**, responsável por instanciar os nós do JavaFX (botões, *canvas*) e construir a árvore visual.
3. Invocação do método **`registerHandlers()`**, responsável por associar os *listeners* a eventos (rato, teclado) e associar o padrão Observer (**`PropertyChangeSupport`**) ao modelo, instruindo-o sobre quando o ecrã deve ser atualizado.
4. Invocação do método **`update()`**, atualiza proativamente o estado inicial do ecrã consultando métodos do tipo **`get`** na *Facade* do modelo.

Diagrama de Estados (FSM)

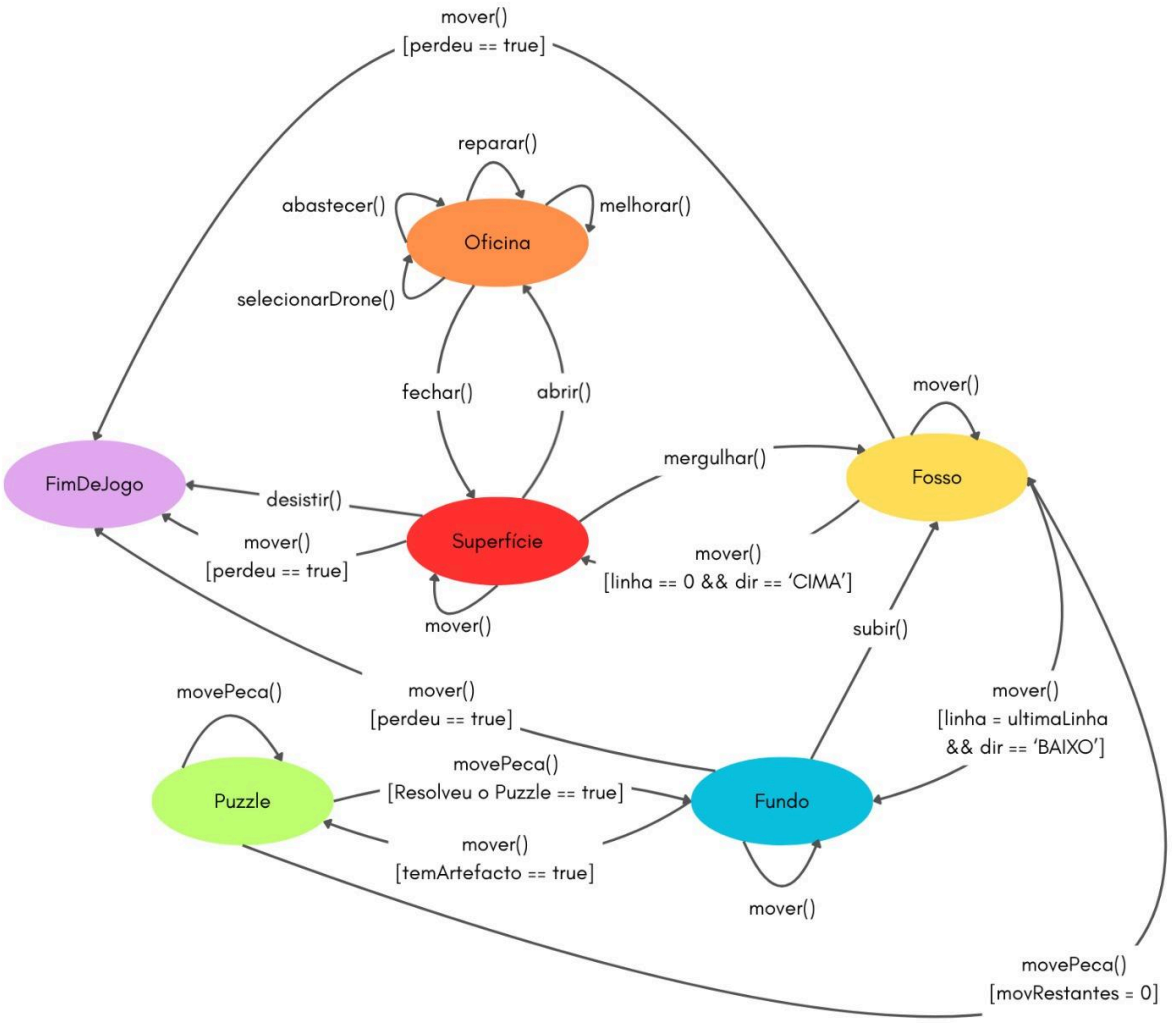


Diagrama de Classes e Estrutura de Pacotes

Estrutura de Pacotes

`pt.isec.pa.deepsea.model.data`: Contém todas as entidades ativas (**`Navio`**, **`Drone`**), Elementos passivos (**`Rocha`**, **`Minerio`**, **`Artefacto`**) e as estruturas espaciais (**`Fosso`**, **`Fundo`**, **`Superficie`**, **`Celula`**). Faz forte uso de herança a partir da classe **`Elemento`**.

`pt.isec.pa.deepsea.model.state`: A implementação da Máquina de Estados (State Pattern).

`pt.isec.pa.deepsea.model`: A camada de serviços, contendo o **`DeepSeaManager`** (Facade), o sistema de Logs e Records imutáveis.

`pt.isec.pa.deepsea.ui`: O frontend em JavaFX (View-Controller).

`pt.isec.pa.deepsea.ui.res`: Gestores de cache de recursos multimédia (CSS, MP3, PNG).

Diagramas

Diagrama UML do Model

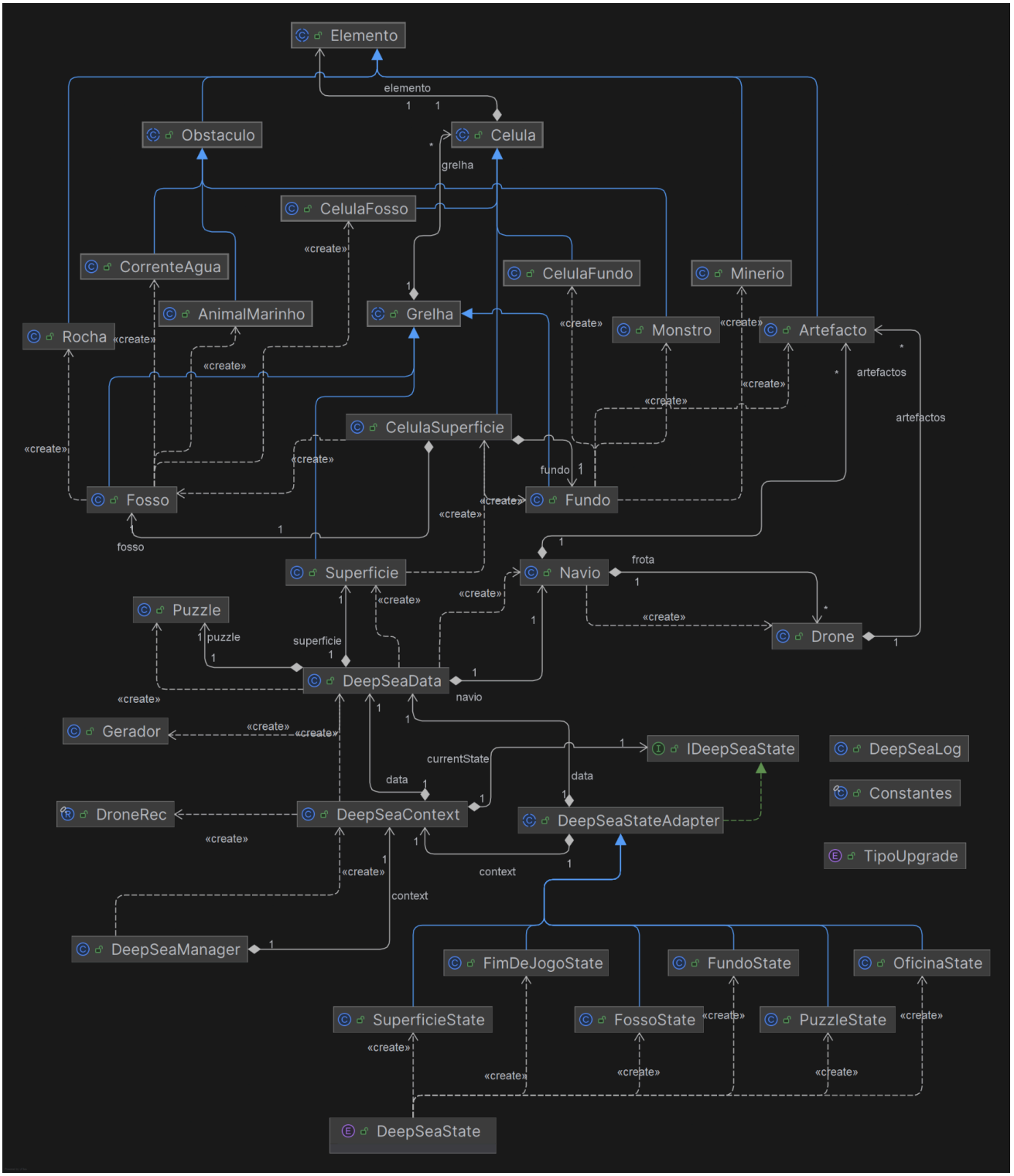


Diagrama UML do UI

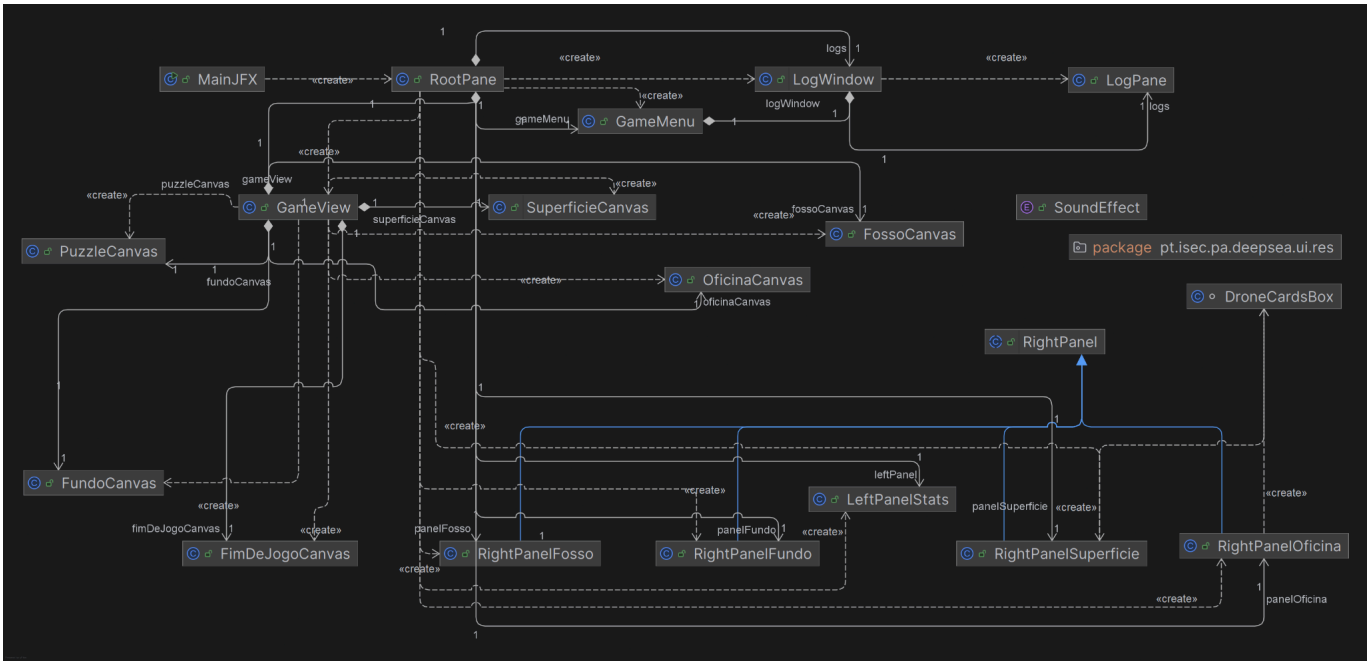


Tabela com Resumo das Tarefas

Etapa 1

Requisito/Funcionalidade	
Estrutura de Classes Base (Jogo, Navio, Drones, Grelhas)	☑
Geração de Fossos e Superfície (Algoritmo de oscilação das rochas)	☑
Geração do Fundo do Mar (Minérios, Artefactos e Monstros)	☑
Lógica do Minijogo	☑
Métodos de Consulta e Ordenação	☑
Modo Defesa	☑
Testes Unitários	☑

Etapa 2

Requisito/Funcionalidade	
Máquina de Estados (Context, State, Adapter e estados concretos)	☑
Facade DeepSeaManager (Encapsulamento e ocultação da FSM)	☑
Persistência de Dados (Gravação/Carregamento via Serialização)	☑
Classe DeepSeaLog (Singleton) (Registo com timestamps)	☑
Métodos de Consulta e Ordenação	☑
Funcionalidades do Log (Consulta, limpeza e exportação para .txt)	☑
Documentação Javadoc Inicial	☑

Etapa 3

Requisito/Funcionalidade	
Interface Gráfica Base	☑
Notificações Assíncronas	☑
Renderização baseada em Canvas e StackPane	☑
Representação Visual (Imagens dos elementos)	☑
Métodos de Consulta e Ordenação	☑
Barra de Menus (MenuBar) (Game e Log)	☑

Etapa 4

Requisito/Funcionalidade	
Gestão de Áudio (SoundManager)	☑
Ativação/Desativação de Som (BooleanProperty)	☑
Redimensionamento Proporcional das Grelhas	☑
Preparação de uma Segunda Janela de UI Sincronizada	☑
Finalização de Testes e Javadoc	☑

Conclusão

A realização do projeto "Deep Sea Mining" foi fundamental para consolidar os conceitos práticos de Programação Orientada aos Objetos e Padrões de Desenho lecionados nas aulas. A abordagem incremental por etapas provou a importância de manter uma arquitetura modular e de baixo acoplamento.

O cumprimento rigoroso do padrão MVC@PA permitiu isolar completamente a lógica de negócio da interface gráfica. Graças a este desacoplamento, a transição para o ambiente JavaFX foi feita de forma fluida, facilitando a implementação de requisitos complexos como a sincronização em tempo real da segunda janela e o redimensionamento responsivo das grelhas. Adicionalmente, as decisões de design do grupo, como a criação de estados dedicados para a Oficina e Fim de Jogo, revelaram-se acertadas para evitar a sobrecarga visual na UI, garantindo uma interface limpa, intuitiva e focada na experiência do utilizador.

Em suma, todos os objetivos estipulados nos enunciados foram superados com sucesso. O resultado final é uma aplicação robusta, totalmente funcional, documentada e pronta para a defesa final do projeto.